# Data Models Query Language

## Relational Model & SQL

Week 2

# Today's Agenda

1. **The Relational Model & SQL**
   - History and Core Concepts (DDL/DML)
   - Defining Schemas: 'CREATE', 'ALTER', 'DROP'
   - Keys and Constraints
2. **Basic Single-Table Queries**
   - The SFW ('SELECT', 'FROM', 'WHERE') block
   - Projection, Selection, and Ordering ('AS', 'DISTINCT', 'ORDER BY')
3. **Aggregation and Grouping**
   - Aggregate Functions ('COUNT', 'SUM', 'AVG', etc.)
   - Grouping with 'GROUP BY' and 'HAVING'
4. **Multi-Table Queries**
   - Foreign Keys and Joins
   - Set Operations ('UNION', 'INTERSECT', 'EXCEPT')

# 1. The Relational Model & SQL

# What is SQL?

- SQL is the standard language for interacting with relational databases.
- It's a very high-level, **declarative** language.
  - You specify *what* data you want, not *how* to get it.
  - The Database Management System (DBMS) is responsible for optimizing the "how" (the query plan).
- Many standards exist, but most systems are compatible with **SQL-92**.
  - ANSI SQL, SQL:1999, SQL:2003, etc.
  - Most vendors (PostgreSQL, MySQL, SQL Server) add their own proprietary extensions.

**SQL stands for**

**Structured Query Language**

# SQL History

- Developed at IBM by Donald Chamberlin and Raymond Boyce in the early 1970s.
- Originally called **SEQUEL** (Structured English Query Language) for the **System R** prototype DBMS.
- Built upon Edgar F. Codd's relational model of data.
- Commercially released by IBM in products like System/38 (1979) and DB2 (1983).
- Became the de-facto standard, outlasting other early query languages like QUEL.

**Is SQL's dominance ending?**

A 2023 tweet by Jo Kristian Bergum claimed: *"Natural query languages (NQL) replace the structured query language (SQL)."* While AI and NQL are powerful, SQL's precision, performance, and deep integration into the data ecosystem ensure its relevance for the foreseeable future.

# The Two Sides of SQL

- **Data Definition Language (DDL)**
  - For defining the database structure or "schema".
  - Commands: 'CREATE', 'ALTER', 'DROP'.
  - Think of it as the *blueprint* for your database.

# The Two Sides of SQL

- **Data Definition Language (DDL)**
  - For defining the database structure or "schema".
  - Commands: 'CREATE', 'ALTER', 'DROP'.
  - Think of it as the *blueprint* for your database.

- **Data Manipulation Language (DML)**
  - For querying and modifying the data itself.
  - Commands: 'SELECT', 'INSERT', 'UPDATE', 'DELETE'.
  - This is how you *interact with the data* stored in your tables.

# What is a Table?

**Students**

| sid | name | major |
|---|---|---|
| 101 | Ann | CS |
| 102 | Bob | EE |
| 103 | Chris | CS |
| 104 | David | BIO |

A **relation** (or table) is a **multiset** of **tuples**, where each tuple conforms to the attributes defined in the schema.

Let's break that down.

# Breaking it Down: Tuples and Attributes

**Students**

| sid | name | major |
|---|---|---|
| 101 | Ann | CS |
| 102 | Bob | EE |
| 103 | Chris | CS |
| 104 | David | BIO |

**Attribute (Column)**

- A typed data field. E.g., 'sid' is an integer, 'name' is a string.
- All values in a column must be of the same type.

**Tuple (Row)**

- A single record or entry in the table.
- The highlighted row is one tuple.

# Breaking it Down: Multiset and Cardinality/Arity

**Students**

| sid | name | major |
|---|---|---|
| 101 | Ann | CS |
| 102 | Bob | EE |
| 103 | Chris | CS |
| 104 | David | BIO |

**Multiset**

- An unordered collection that allows duplicates.
- Unlike a pure mathematical set, a table can have identical rows (though this is bad practice and avoided with keys).
- The order of rows is not guaranteed unless you use 'ORDER BY'.

# Breaking it Down: Multiset and Cardinality/Arity

**Students**

| sid | name | major |
|---|---|---|
| 101 | Ann | CS |
| 102 | Bob | EE |
| 103 | Chris | CS |
| 104 | David | BIO |

**Cardinality and Arity**

- **Cardinality**: The number of tuples (rows). Here, it's 4.
- **Arity**: The number of attributes (columns). Here, it's 3.

# The Logic of Relations

- A relation is a **set** of tuples; a schema is a **set** of attributes.
- **Implication**: The order of rows or attributes is immaterial.
- If you swap the "Name" and "Major" columns, it is still the *same* relation.
- If you shuffle the rows, it is still the *same* relation.
- This allows the DBMS to store data in whatever order is most efficient on the disk.

# Data Types in SQL

- Every attribute must have a data type. Common types include:
- **Strings:**
  - ‘CHAR(n)‘: Fixed-length string of ‘n‘ characters. Padded with spaces.
  - ‘VARCHAR(n)‘: Variable-length string up to ‘n‘ characters.
  - ‘TEXT‘: For long-form text.
- **Numbers:**
  - ‘INT‘ or ‘INTEGER‘, ‘BIGINT‘, ‘SMALLINT‘.
  - ‘DECIMAL(p, s)‘: Exact numeric type with precision ‘p‘ and scale ‘s‘. Good for money!
  - ‘FLOAT‘, ‘REAL‘: Floating-point numbers.
- **Date/Time:**
  - ‘DATE‘: Stores year, month, and day.
  - ‘TIME‘: Stores hour, minute, second.
  - ‘TIMESTAMP‘: Stores both date and time.

# Table Schemas and Keys

- The **schema** is the formal definition of a table: its name, attributes, and their types.

### Schema Notation

Students(sid: INT, name: VARCHAR(50), major: VARCHAR(30))

- A **key** is an attribute (or set of attributes) whose value uniquely identifies each tuple. We underline the key in our notation.

### Schema with Primary Key

Students(<u>sid</u>: INT, name: VARCHAR(50), major: VARCHAR(30))

# Types of Keys

A key is a **minimal set of attributes** that uniquely identifies a tuple.

- **Superkey**: Any set of attributes that uniquely identifies a tuple.
  - E.g., ‘sid‘ is a superkey. ‘sid, name‘ is also a superkey (but it’s not minimal).
- **Candidate Key**: A *minimal* superkey. You cannot remove any attribute from it and have it still be a unique identifier.
  - In our ‘Students‘ table, ‘sid‘ is a candidate key.
  - A table can have multiple candidate keys (e.g., ‘email‘).
- **Primary Key**: The *one* candidate key chosen by the database designer to be the main identifier for the table.

# Constraints: NULL and NOT NULL

- ‘NULL‘ is a special marker used to represent a missing or unknown value.
- It is not the same as zero or an empty string!
- We can constrain an attribute to forbid ‘NULL‘ values. This is essential for primary keys.

**Students(sid, name, gpa)**

| sid | name | gpa |
|---|---|---|
| 101 | Ann | 3.9 |
| 105 | Eve | NULL |

Eve just enrolled and doesn’t have a GPA yet.

In SQL, we would declare ‘sid‘ and ‘name‘ as ‘NOT NULL‘.

```
CREATE TABLE Students (
    sid INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    gpa DECIMAL(3, 2)
```

# General Constraints & Performance

- SQL allows arbitrary assertions (e.g., "There cannot be 90 people in this class").
- **The Architect's Trade-off**: In practice, we rarely use complex general constraints.
- **Why?** Performance! Every constraint must be checked during every `INSERT` or `UPDATE`.
- Whenever a database system avoids a "convenient" feature, it is usually for the sake of speed.

# The Challenge of NULL: Three-Valued Logic

Because ‘NULL‘ means "unknown", it introduces complexity into comparisons. SQL uses three-valued logic (3VL).

- **TRUE**
- **FALSE**
- **UNKNOWN**

Consider the expression ‘gpa $>$ 3.0‘.

- For Ann (‘gpa $=$ 3.9‘), the result is **TRUE**.
- For Eve (‘gpa $=$ NULL‘), the result is **UNKNOWN**. We don't know if her GPA is greater than 3.0 or not.

**Important!**

The ‘WHERE‘ clause of a query only keeps rows where the condition evaluates to **TRUE**. Rows that evaluate to ‘FALSE‘ or ‘UNKNOWN‘ are discarded.

To check for ‘NULL‘ itself, you must use ‘IS NULL‘ or ‘IS NOT NULL‘. ‘gpa $=$ NULL‘ will always be ‘UNKNOWN‘.

# Creating and Modifying Tables (DDL)

- Use ‘CREATE TABLE‘ to define a new table.

```
CREATE TABLE Courses (
    cid          CHAR(10) PRIMARY KEY,
    title        VARCHAR(100) NOT NULL,
    credits      INT,
    department   VARCHAR(30)
);
```

- Use ‘ALTER TABLE‘ to modify an existing one, and ‘DROP TABLE‘ to delete it.

```
-- Add a new column to the table
ALTER TABLE Courses ADD prereq CHAR(10);

-- Delete the entire table and all its data!
DROP TABLE Courses;
```

# Populating Tables (DML)

- ‘INSERT‘ adds new tuples (rows).

```
INSERT INTO Students (sid, name, major)
VALUES (101, 'Ann', 'CS');

-- If providing all values in order, column list is optional
INSERT INTO Students VALUES (102, 'Bob', 'EE', 3.5);
```

- ‘DELETE‘ removes tuples that match a condition.

```
-- Remove all students with a student ID greater than 200
DELETE FROM Students WHERE sid > 200;
```

- ‘UPDATE‘ modifies attributes of existing tuples.

```
-- Change Bob's major to Computer Engineering
UPDATE Students
SET major = 'CprE', gpa = 3.6
WHERE sid = 102;
```

# 2. Basic Single-Table Queries

# The SFW Query Block

- The fundamental structure of almost every SQL query.

```
SELECT    <attributes to return>
FROM      <table(s) to query>
WHERE     <condition to filter rows>
```

**Conceptual Order of Operations:**

1. **FROM**: Identify the source table.
2. **WHERE**: Filter rows based on the condition.
3. **SELECT**: Choose which columns (and expressions) to output.

This is not how the DBMS actually runs the query, but it's the correct way to think about the logic.

# Example: Selection

- The ‘WHERE‘ clause performs **selection**, filtering the rows.
- ‘*‘ in the ‘SELECT‘ list means "all attributes".

**Students(<u>sid</u>, name, major, gpa)**

*Query: Find all information about Computer Science students.*

```
SELECT *
FROM Students
WHERE major = 'CS';
```

**Result:**

| sid | name | major | gpa |
|---|---|---|---|
| 101 | Ann | CS | 3.9 |
| 103 | Chris | CS | 3.2 |

# Example: Projection

- The ‘SELECT‘ clause performs **projection**, choosing which columns to display.
- You can rename columns using ‘AS‘ (aliasing).

*Query: Find the name and GPA of all CS students.*

```
SELECT name, gpa AS grade_point_average
FROM Students
WHERE major = 'CS';
```

**Result:**

| name | grade_point_average |
|---|---|
| Ann | 3.9 |
| Chris | 3.2 |

# Projection with Expressions

- The ‘SELECT‘ list can contain expressions, not just column names.

*Query: Show each student’s name and their GPA on a 100-point scale.*

```
SELECT name, (gpa * 25) AS gpa_100_scale
FROM Students;
```

**Result:**

| name | gpa_100_scale |
|---|---|
| Ann | 97.5 |
| Bob | 90.0 |
| Chris | 80.0 |
| David | 92.5 |

# DISTINCT and ORDER BY

- ‘DISTINCT‘ eliminates duplicate rows from the result.
- ‘ORDER BY‘ sorts the final result set. ‘ASC‘ (default) or ‘DESC‘.

*Query: List all unique majors.*

```
SELECT DISTINCT major
FROM Students;
```

**Result:**

- CS
- EE
- BIO

*Query: List all students, ordered by GPA descending.*

```
SELECT name, gpa
FROM Students
ORDER BY gpa DESC;
```

**Result:**

- Ann, 3.9
- David, 3.7
- Bob, 3.6
- Chris, 3.2

# More Complex WHERE Clauses

The ‘WHERE‘ clause can combine conditions with ‘AND‘, ‘OR‘, and ‘NOT‘. You can also use pattern matching with ‘LIKE‘.

- ‘%‘ matches any sequence of zero or more characters.
- ‘_‘ matches any single character.

*Query: Find all students whose name starts with ’A’ or who have a GPA below 3.5.*

```
SELECT name, major, gpa
FROM Students
WHERE name LIKE 'A%' OR gpa < 3.5;
```

**Result:**

| name | major | gpa |
|---|---|---|
| Ann | CS | 3.9 |
| Chris | CS | 3.2 |

# String Comparison Details

- **Case Sensitivity**:
  - SQL keywords are *case-insensitive* (`SELECT` = `select`).
  - String values are *case-sensitive* ('Seattle' $\neq$ 'seattle').
- **Fixed-length CHAR(n)**: If a string has fewer than *n* characters, it is padded with spaces. Most systems ignore these trailing spaces during equality checks.
- **Lexicographic Order**: Comparison operators (`<`, `>`, `<=`, `>=`) use dictionary order.
  - `'fodder' < 'foo'` is **TRUE**.
  - `'bar' < 'bargain'` is **TRUE**.

# 3. Aggregation & Grouping

# Aggregate Functions

- Aggregates compute a single value from a set of rows. They are used in the ‘SELECT‘ list.

| Function | Description |
|---|---|
| ‘COUNT(*)‘ | Counts the number of rows. |
| ‘COUNT(col)‘ | Counts non-NULL values in ‘col‘. |
| ‘SUM(col)‘ | Computes the sum of numeric values. |
| ‘AVG(col)‘ | Computes the average of numeric values. |
| ‘MIN(col)‘ | Finds the minimum value. |
| ‘MAX(col)‘ | Finds the maximum value. |

*Query: Find the total number of students and their average GPA.*

```
SELECT COUNT(*) AS num_students, AVG(gpa) AS avg_gpa
FROM Students;
```

# Grouping with GROUP BY

- What if we want to apply aggregates to *subsets* of the data?
- The 'GROUP BY' clause groups rows that have the same values in specified columns into summary rows.
- Aggregates are then applied to each group individually.

*Query: Find the number of students and average GPA for each major.*

```
SELECT major, COUNT(*) AS num_in_major, AVG(gpa) AS avg_gpa
FROM Students
GROUP BY major;
```

**Result:**

| major | num_in_major | avg_gpa |
|---|---|---|
| CS | 2 | 3.55 |
| EE | 1 | 3.60 |
| BIO | 1 | 3.70 |

# Filtering Groups with HAVING

- The ‘WHERE‘ clause filters rows *before* they are grouped.
- The ‘HAVING‘ clause filters *groups* after aggregation has occurred.

**Key Idea**

Use ‘WHERE‘ for conditions on individual rows. Use ‘HAVING‘ for conditions on aggregate values.

*Query: Find majors where the average GPA is greater than 3.6.*

```
SELECT major, AVG(gpa) AS avg_gpa
FROM Students
GROUP BY major
HAVING AVG(gpa) > 3.6;
```

**Result:**

| major | avg_gpa |
|---|---|
| BIO | 3.70 |

# 4. Multi-Table Queries

# Foreign Keys

A **foreign key** is a column (or set of columns) in one table that uniquely identifies a row of another table. It creates a link between them.

**Students(sid, name, major)** **Enrollment(student_sid, course_cid, grade)**

- The 'student_sid' column in the 'Enrollment' table *refers to* the 'sid' column in the 'Students' table.
- This enforces **referential integrity**: you cannot enroll a student who does not exist in the 'Students' table.

```
CREATE TABLE Enrollment (
    student_sid INT,
    course_cid  CHAR(10),
    grade       CHAR(2),
    PRIMARY KEY (student_sid, course_cid),
    FOREIGN KEY (student_sid) REFERENCES Students(sid)
);
```

# Combining Tables with Joins

- A ‘JOIN‘ combines rows from two or more tables based on a related column between them.
- The ‘FROM‘ clause lists the tables, and the ‘WHERE‘ clause specifies the join condition.

*Query: Find the names of students and the course IDs they are enrolled in.*

```
SELECT s.name, e.course_cid
FROM Students AS s, Enrollment AS e
WHERE s.sid = e.student_sid; -- This is the join condition
```

## Modern Syntax

It’s better practice to use the explicit ‘JOIN‘ keyword.

```
SELECT s.name, e.course_cid
FROM Students AS s
JOIN Enrollment AS e ON s.sid = e.student_sid;
```

# Resolving Variable Ambiguity

If two tables share a column name (e.g., `name`), the query will fail with an "ambiguous column" error.

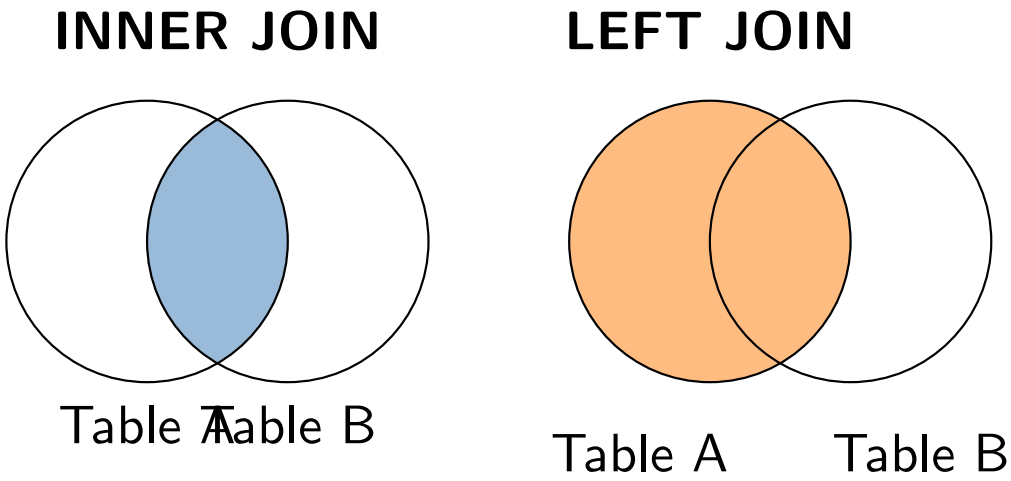
**Problematic Query:**

```
SELECT name, cid FROM Students, Enrollment WHERE sid =
    student_sid;
-- Which "name"? Students.name or Enrollment.name?
```

**Solution: Dot Notation and Aliasing**

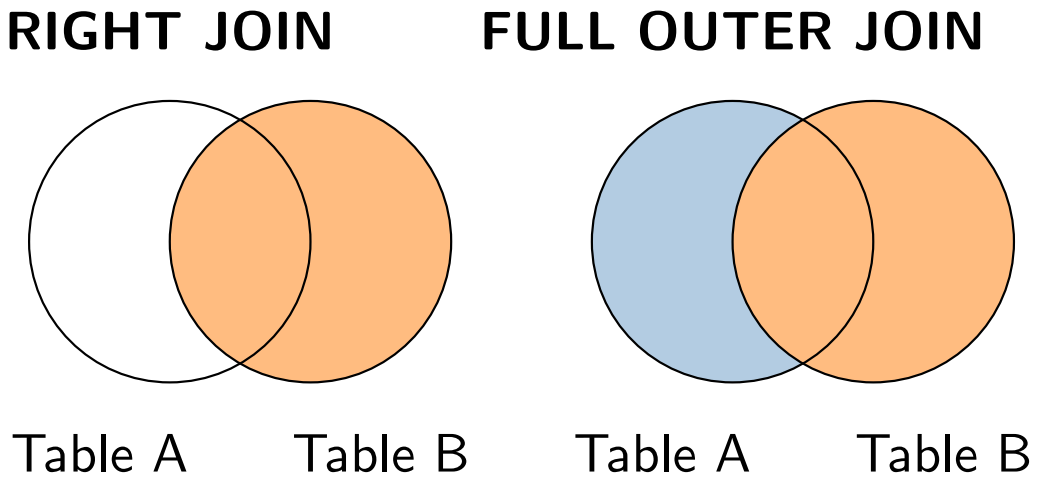
```
-- Explicitly naming the table
SELECT Students.name, Enrollment.cid FROM Students,
    Enrollment...

-- Using range variables (aliases)
SELECT s.name, e.cid
FROM Students s, Enrollment e
WHERE s.sid = e.student_sid;
```

# Visualizing Join Types




- **INNER JOIN**: Returns records that have matching values in both tables.
- **LEFT JOIN**: Returns all records from the left table, and the matched records from the right table. Unmatched right-side records are 'NULL'.

# Visualizing Join Types




- **RIGHT JOIN**: Returns all records from the right table, and the matched records from the left table.
- **FULL OUTER JOIN**: Returns all records when there is a match in either left or right table.

# Conceptual Semantics of a Query

How should you think about what a query does?

```
SELECT S.name, E.grade
FROM Students S, Enrollment E
WHERE S.sid = E.student_sid AND S.major = 'CS';
```

1. **Cross Product (FROM)**: Imagine a giant, temporary table containing every possible combination of rows from 'Students' and 'Enrollment'.
2. **Selection (WHERE)**: Go through the giant table row-by-row and keep only the rows that satisfy the 'WHERE' condition (e.g., where 'sid' matches 'student_sid' and 'major' is 'CS').
3. **Projection (SELECT)**: From the rows that you kept, pull out only the columns specified in the 'SELECT' list.

**T**

his is the **logical meaning**, not the physical execution plan! The DBMS uses indexes and other tricks to avoid actually building the giant cross product.

# Set Operations

SQL can combine the results of two queries using set operations. The queries must have the same number and types of columns.

- **UNION**: Combines results and removes duplicates. ‘UNION ALL‘ keeps duplicates.
- **INTERSECT**: Returns only rows that appear in both query results.
- **EXCEPT** (or ‘MINUS‘ in some systems): Returns rows from the first query that are not in the second.

*Query: Find students who are in CS or have a GPA ¿ 3.8.*

```
-- Query 1: CS Students
SELECT sid, name FROM Students WHERE major = 'CS'
UNION
-- Query 2: High-achieving students
SELECT sid, name FROM Students WHERE gpa > 3.8;
```